

High Performance Programming [COMTEK3, & ESD3]
Lecture 4: Analytical Modelling of Parallel Systems and Applications

03/2025

Instruction: *The exercise problems are designed to aid your understanding of the concepts covered during the lecture. They are intended to be discussed and solved in a group of 3 – 6 students. The most important part of the exercise is the **learning process**. It is strongly recommended that you avoid the usage of LLMs such as chatGPT and similar AI models to generate solutions **without trying to solve the problems first**.*

Exercise 1: Efficiency of Parallel Sum Algorithm

- (a) Complete the Table 1 for the efficiency of the sum algorithm for adding n numbers on p processing units.
- (b) Make a plot of the speedup and efficiency as a function of the number of processes for the different input sizes.
- (c) Using a balanced binary tree, illustrate the computation performed by the work-efficient parallel prefix sum on $A = [5, 8, -6, 4, 7, 17, -2, 4]$.

n	$p = 1$	$p = 4$	$p = 8$	$p = 16$	$p = 32$
16					
32					
64					
128					
256					
512					

Table 1: Efficiency of sum algorithm

Solution:

- (a) For the sum algorithm, the sequential running time is

$$T_{seq} = n - 1,$$

and the parallel running time (assuming equal partitioning of the data among p processes) is

$$T_p = \left(\frac{n}{p} - 1 \right) + \log_2(p).$$

The first term of T_p accounts for the local summation at each process and the second term corresponds to the parallel reduction of p partial sums to obtain the final sum.

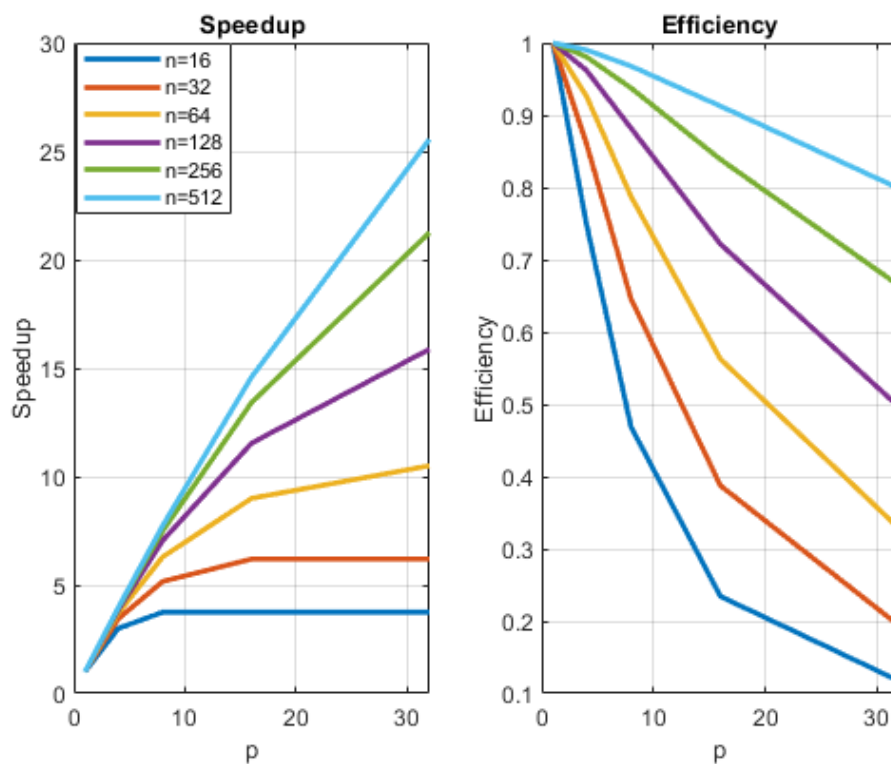
If $p \geq n$ (or $\frac{n}{2}$), then no partition is required and a schedule can be found such that

$$T_p = \log_2(n)$$

Using the definition of speedup and efficiency, we obtain the efficiency in the table below and the plots in (b) .

n	$p = 1$	$p = 4$	$p = 8$	$p = 16$	$p = 32$
16	1	0.75	0.47	0.23	0.11
32	1	0.86	0.65	0.39	0.19
64	1	0.93	0.79	0.56	0.33
128	1	0.96	0.88	0.72	0.50
256	1	0.98	0.94	0.84	0.66
512	1	0.99	0.98	0.91	0.80

(b)



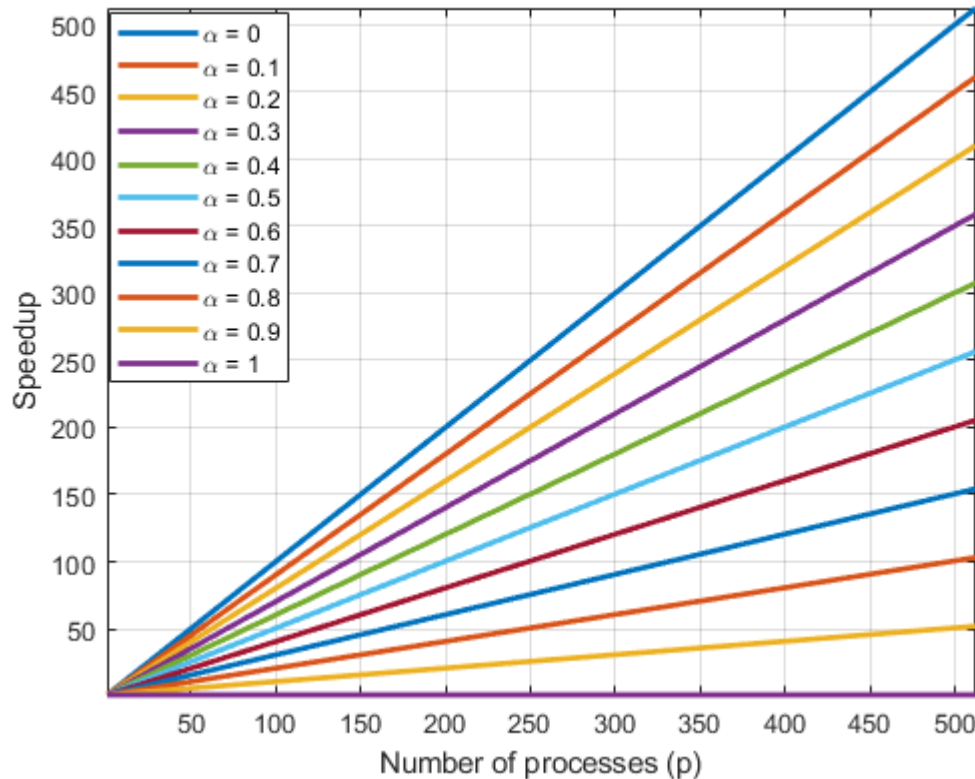
(c). The solution is obtained by replacing $a_1 - a_8$ in the example on slide 28 with the entries of A .

Exercise 2: Speedup using Gustafson's Law

- Using any programming language of your choice, plot the speedup as a function of the number of processors using Gustafson's Law for $\alpha \in \{0, 0.1, 0.2, \dots, 1\}$.
- Comment on your observation.

Solution: (a) Gustafson's Law states that when you scale up the data set with the number of processors, the overall speedup behaves as

$$S_{Gustafson} = p - \alpha(p - 1)$$



(b) What do you observe from the figure about the behaviour of speedup with increasing p , n and α ? [Add here!]

Exercise 3: Asymptotic Comparison of Algorithms

Consider four algorithms for sorting a given list of n numbers as shown in Table 2.

- Which of these algorithms do you consider 'best' and why?
- Which of these are scalable and which are not?

Solution:

(a) Parallel algorithms A_2 and A_4 are considered the best because they have the lowest "cost" (processor-time product) asymptotically. See the table for a ranking based on the cost.

(b) A scalable parallel algorithm is one that is able to keep efficiency constant. A_2 and A_4 are the most scalable algorithms.

Algorithm	A_1	A_2	A_3	A_4
p	n^2	$\log_2(n)$	n	$\sqrt{n}$
T_p	1	n	$\sqrt{n}$	$\sqrt{n} \log_2(n)$
S	$n \log_2(n)$	$\log_2(n)$	$\sqrt{n} \log_2(n)$	$\sqrt{n}$
E_p	$\frac{\log_2(n)}{\sqrt{n}}$	1	$\frac{\log_2(n)}{n}$	1
pT_p	$\frac{n^2}{\sqrt{n}}$	$n \log_2(n)$	$n^{1.5}$	$n \log_2(n)$
Ranking	3	1	2	1

Table 2: Asymptotic comparison of algorithms

Exercise 4: Superlinear speedup

Consider the search trees shown in Figure 1, in which the red node represents the solution.

- If a sequential search of the tree is performed using the standard depth-first search (DFS) algorithm, how much time does it take to find the solution if traversing each arc of the tree takes one unit of time?
- Assume that the tree is partitioned between two processing elements that are assigned to do the search job, as shown in Figure 1 (right). If both processing elements perform a DFS on their respective halves of the tree, how much time does it take for the solution to be found? What is the speedup? Is there a speedup anomaly? If so, can you explain the anomaly?
- Repeat (a) and (b) for the tree in the Figure 2.

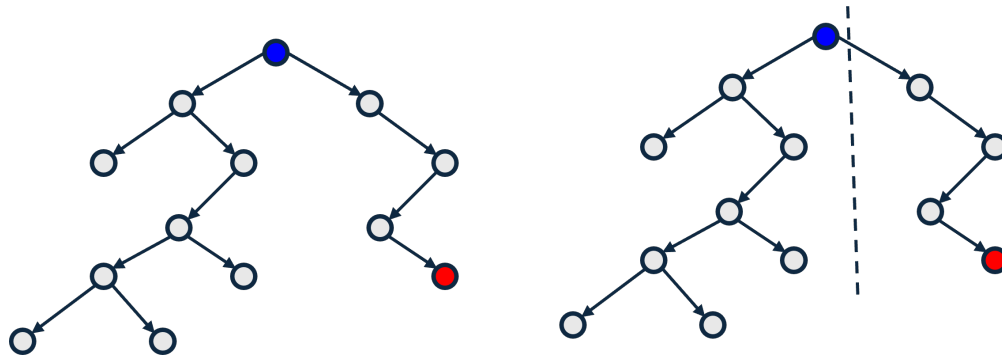


Figure 1: Depth First Search with one processor (left) and 2 processors (right)

Solution:

- (a) The sequential algorithm takes:

$$T_1 = 12t_c$$

- (b) With 2 processors, the running time is

$$T_2 = 3t_c,$$

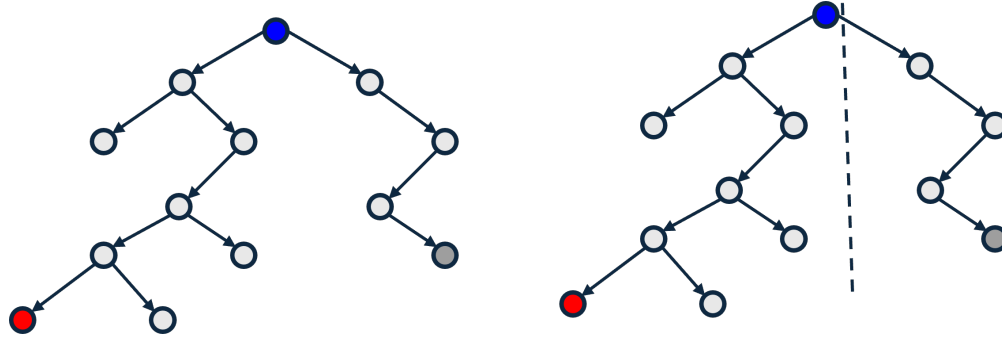


Figure 2: Depth First Search with one processor (left) and 2 processors (right)

and the speed is

$$S = \frac{T_1}{T_2} = 12/3 = 4$$

The speedup is superlinear since $4 > 2$. This anomaly depends largely on the partition and the relative location of the target node (red) to the root (blue).

(c) For the tree in Figure 3, we have

$$T_1 = T_2 = 6t_c \rightarrow S = \frac{6}{6} = 1$$

No speedup is achieved with 2 processors and parallel DFS searches more nodes than its sequential counterpart.

Exercise 5: Scaled Speedup

Scaled speedup is defined as the speedup obtained when the problem size is increased linearly with the number of processing elements. That is, if W is chosen as a base problem size for a single processing element, then:

$$S_{scaled} = \frac{pW}{T_p(pW, p)}$$

- Consider the problem of adding n numbers on p processing elements, and plot the speedup curves, assuming that the base problem for $p = 1$ is that of adding 256 numbers. Use $p = 1, 4, 16, 64, 256$.
- Assume that it takes 10-time units to communicate a number between two processing elements and that it takes one unit of time to add two numbers. Now plot the standard speedup curve for the base problem size and compare it with the scaled speedup curve (Hint: The parallel runtime $T_p = (n/p - 1) + 11 \log_2(p)$).

Solution: (a) If the base problem is $n = 256$ for 1 processor, then for p processors the new problem size is $n' = 256p$.

Assuming equal partitioning of the data to be summed among p processors, the parallel time is

$$T_p = \left(\frac{256p}{p} - 1 \right) + \log_2(p) = 255 + \log_2(p)$$

The scaled speedup is therefore:

$$S_{scaled} = \frac{256p}{255 + \log_2(p)}$$

Add plot of the above expression here.

(b) Standard speedup:

$$T_1(n = 256) = n - 1 = 255; \quad T_p(256, p) = \left(\frac{n}{p} - 1 \right) + 11 \log_2(p) = \left(\frac{256}{p} - 1 \right) + 11 \log_2(p)$$

$$S_{standard} = \frac{255}{\left(\frac{256}{p} - 1 \right) + 11 \log_2(p)}$$

For scaled speedup, we have

$$T_p(256p, p) = \left(\frac{255p}{p} - 1 \right) + 11 \log_2(p) = 255 + 11 \log_2(p)$$

$$S_{scaled} = \frac{255p}{255 + 11 \log_2(p)}$$

Insert the plot here.

Exercise 6: Application of Parallel Prefix Scan

Parallel prefix scan has many applications in parallel computing.

- Describe how parallel prefix scan can be used in computing the exclusive scan of an array.
- Explain how parallel prefix scan can be applied to solve the problem of parallel radix sorting.
- Discuss how parallel prefix scan can be used to efficiently compute the histogram of a large dataset in parallel.

Solution:

(a) **Algorithm: Parallel Exclusive Scan**

1. Initialize an array A of size n .
2. Perform an *up-sweep* (reduce phase):
 - (a) Iteratively sum pairs of elements in a tree-like fashion until a single sum is obtained.
3. Perform a *down-sweep* (distribution phase):
 - (a) Replace the last element with zero and propagate the results down the tree.
4. Return the resulting exclusive scan array.

(b) Algorithm: Parallel Prefix Scan for Radix Sort

1. For each digit position (least to most significant):
 - (a) Compute digit counts in parallel.
 - (b) Compute a parallel prefix scan on the counts to determine positions.
 - (c) Scatter elements into their sorted positions using parallel processing.
2. Repeat for the next significant digit until all are processed.

(c) Algorithm: Parallel Histogram Computation

1. Split the dataset among the processors and compute the local frequency count of elements in each bin in parallel.
2. Perform a parallel prefix scan on the histogram counts to form a global prefix.
3. Use the prefix sum results to compute cumulative bin counts.

RAD